

PROJECT 1 Medium Level

Event Registration Platform

Stack	Spring Boot, Spring Security (JWT), Spring Data JPA, MySQL, Redis, JavaMailSender
Architecture	Layered REST API — Controller → Service → Repository
Difficulty	Beginner-friendly setup, non-trivial business logic under the hood

Overview

A backend system for managing events end-to-end — from creation and capacity control to registrations, ticketing, and automated notifications. The complexity here is not in the CRUD but in the edge cases: concurrent registrations fighting over the last seat, waitlist promotion logic, cancellation policies with partial refund windows, and role-gated access across organiser and attendee contexts.

Core Modules

1. User & Auth Module

- JWT-based authentication with refresh token rotation
- Role-based access control: ADMIN, ORGANISER, ATTENDEE
- Email verification on signup via tokenized link (15-minute expiry)
- Password reset flow with single-use token stored in Redis

2. Event Management Module

- Organiser can create events with: title, description, venue, date/time, max capacity, ticket price
- Event states: DRAFT → PUBLISHED → ONGOING → COMPLETED / CANCELLED
- Organisers can publish, unpublish, or cancel events — cancellation triggers refund eligibility flag
- Admin can feature or suppress any event platform-wide
- Soft delete on events; historical data is retained for reporting

3. Registration & Capacity Module

- Atomic seat reservation using database-level row locking (SELECT FOR UPDATE) to prevent overselling
- Waitlist auto-enrolment when event is full; configurable waitlist cap per event
- On cancellation by a registered attendee, the next waitlisted user is automatically promoted
- Duplicate registration guard — one active registration per user per event
- Team registration support: one member registers a group, all receive individual tickets

4. Ticketing Module

- Unique QR-code token generated per confirmed registration (UUID-based)
- Ticket status: CONFIRMED, CANCELLED, USED

- Check-in endpoint for organisers: scans token, marks as USED — idempotent (double scan returns 200, not error)
- Ticket download endpoint returns ticket metadata; PDF generation can be plugged in

5. Notification Module

- Async email notifications via Spring's `@Async` + `JavaMailSender`
- Triggers: registration confirmed, waitlist joined, waitlist promoted, event cancelled, 24-hour reminder
- Reminder job runs via `@Scheduled` cron — queries events starting in next 24 hours and batches emails

6. Admin & Analytics Module

- Admin dashboard APIs: total registrations per event, check-in rate, waitlist count
- Revenue summary per organiser (price × confirmed registrations)
- Bulk cancel event with notification to all registered attendees

Non-Trivial Engineering Decisions

- Concurrency: naive save-check-save pattern causes race conditions at scale; row-level locking in the reservation transaction prevents overselling without distributed locks
- Waitlist promotion is synchronous within the cancellation transaction — ensures exactly one promotion per cancellation
- Redis for token storage (password reset, email verify): automatic TTL expiry without scheduled cleanup jobs
- Idempotent check-in: prevents double-scan errors at event entry without adding extra state
- Async email with retry: `@Async` keeps the registration response fast; failed emails log to a retry queue (DB table) for a scheduled reprocessor

Key API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register user, send email verification
POST	/api/auth/login	Login, return JWT + refresh token
POST	/api/events	Organiser creates a new event (DRAFT)
PATCH	/api/events/{id}/publish	Organiser publishes event
POST	/api/events/{id}/register	Attendee registers; handles capacity + waitlist
DELETE	/api/registrations/{id}	Cancel registration; triggers waitlist promotion
POST	/api/tickets/{token}/checkin	Organiser checks in attendee via QR token
GET	/api/admin/events/{id}/stats	Admin fetches registration analytics

Database Schema (Core Tables)

- users — id, name, email, password_hash, role, is_verified, created_at
- events — id, organiser_id, title, venue, event_date, max_capacity, price, status, created_at

- registrations — id, user_id, event_id, status (CONFIRMED/WAITLISTED/CANCELLED), registered_at
- tickets — id, registration_id, token (UUID), status (CONFIRMED/USED/CANCELLED), issued_at
- team_registrations — id, lead_user_id, event_id, member_count, group_token
- email_retry_queue — id, recipient, subject, body, attempts, next_retry_at, status

PROJECT 2 HARD Level

Crypto Trading Simulator

Stack	Spring Boot, WebSocket (STOMP over SockJS), Spring Security (JWT), Spring Data JPA, MySQL, Redis, Binance Public WebSocket API
Architecture	REST API + WebSocket server; event-driven price propagation
Difficulty	Real-time systems, WebSocket session management, financial precision, portfolio consistency

Overview

A paper-trading platform where users receive a virtual dollar balance and can simulate buying and selling cryptocurrencies at real-time market prices. The backend connects to Binance's public WebSocket stream to receive live price ticks, broadcasts them to connected clients via STOMP, and processes trade orders against the latest price with financial-grade precision. All portfolio state, trade history, and PnL tracking are persisted and consistent.

Core Modules

1. Auth Module

- JWT authentication with role-based access (USER, ADMIN)
- Each new user receives a configurable starting balance (default: \$10,000 virtual USD)
- Refresh token rotation; stateless session

2. Real-Time Price Feed Module

- On startup, a Spring @Component connects to Binance's public multi-stream WebSocket endpoint
- Subscribes to ticker streams for configured trading pairs (BTC/USDT, ETH/USDT, etc.)
- Incoming price ticks are parsed and stored in a ConcurrentHashMap as the latest price registry — O(1) reads
- A Redis pub/sub channel receives each tick; the Spring WebSocket message broker broadcasts to STOMP topic /topic/prices/{symbol}
- Clients subscribe to individual symbol topics; no polling required
- Price registry is the source of truth for order execution — prevents stale price exploitation

3. Portfolio Module

- Tracks each user's virtual USD balance and crypto holdings
- Holdings stored as: user_id, symbol, quantity (BigDecimal for precision)
- GET /api/portfolio returns current holdings + real-time valuation using live price registry
- Portfolio value calculated server-side on request: $\text{sum}(\text{quantity} \times \text{current_price}) + \text{cash_balance}$

- Total PnL = current portfolio value – initial balance

4. Trade Execution Module

- BUY order: deducts USD (price × quantity), credits holding — atomic transaction
- SELL order: deducts holding quantity, credits USD — validates sufficient holding before execution
- Execution price is fetched from the in-memory price registry at time of request
- Insufficient funds or holding triggers a structured error response (not a generic 400)
- All trades use BigDecimal arithmetic — no floating-point errors on financial values
- Each trade creates an immutable trade record: user, symbol, type, quantity, price, timestamp

5. Watchlist Module

- Users can add/remove symbols to a personal watchlist
- GET /api/watchlist returns symbols with their current live prices from the registry
- No polling: watchlist page subscribes to /topic/prices/{symbol} via WebSocket for each watched symbol
- Watchlist persisted in DB; survives session disconnects

6. Trade History & Analytics Module

- Paginated trade history endpoint: filterable by symbol, type (BUY/SELL), date range
- Realised PnL per symbol: calculated from matched BUY/SELL pairs using FIFO cost basis
- Leaderboard endpoint: ranks all users by current total portfolio value (computed on query, cached in Redis for 30 seconds)

7. Admin Module

- Configure active trading pairs (add/remove symbols — triggers Binance subscription update)
- Reset a user's portfolio to initial state
- View platform-wide trade volume and active WebSocket session count

WebSocket Communication Design

Channel / Endpoint	Purpose
/topic/prices/{symbol}	Server → Client: live price tick for a symbol
/topic/portfolio/{userId}	Server → Client: portfolio value update after a trade
/app/subscribe	Client → Server: subscribe to a specific symbol's feed
/user/queue/trades	Server → Client (private): trade execution confirmation

Key API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register user, allocate virtual balance
POST	/api/trade/buy	Execute a BUY order at live price
POST	/api/trade/sell	Execute a SELL order at live price
GET	/api/portfolio	Get holdings + live valuation + PnL

GET	/api/trades	Paginated trade history with filters
POST	/api/watchlist/{symbol}	Add symbol to watchlist
DELETE	/api/watchlist/{symbol}	Remove symbol from watchlist
GET	/api/watchlist	Get watchlist with live prices
GET	/api/leaderboard	Ranked list of users by portfolio value
GET	/api/prices/{symbol}	Latest price for a symbol (REST fallback)

Database Schema (Core Tables)

- users — id, email, password_hash, role, virtual_balance (DECIMAL 18,8), initial_balance, created_at
- holdings — id, user_id, symbol, quantity (DECIMAL 18,8), avg_buy_price (DECIMAL 18,8)
- trades — id, user_id, symbol, trade_type (BUY/SELL), quantity, price, total_value, executed_at
- watchlist — id, user_id, symbol, added_at
- price_snapshots — id, symbol, price, recorded_at (optional; for charting history)

Non-Trivial Engineering Decisions

- BigDecimal throughout: all monetary and quantity fields use DECIMAL(18,8) in MySQL and BigDecimal in Java — no floating-point drift on financial calculations
- Atomic trades with @Transactional: balance deduction and holding credit happen in one transaction; a failure rolls back both
- In-memory price registry: ConcurrentHashMap<String, BigDecimal> holds latest prices; updated by the Binance WebSocket listener thread — trade execution reads from this map, not the DB, for speed and consistency
- WebSocket session management: Spring keeps track of STOMP subscriptions; on disconnect, subscriptions are cleaned up automatically — no manual session registry needed
- Leaderboard caching: computing portfolio value for all users on every request is expensive; Redis caches the ranked list for 30 seconds, invalidated on any trade
- Binance reconnect logic: Binance drops idle WebSocket connections; a @Scheduled heartbeat pings the connection and reconnects if dropped — production-grade resilience